

car-coating ユーザーマニュアル

プロダクト名: car-coating (Keeper PRO SHOP マルチテナント店舗サイト + 管理ビルダー) **バージョン:** 0.1.0 **日付:** 2026-05-07 **概要:** Keeper PRO SHOP のコーティング店舗向け、Next.js ベースのマルチテナント店舗サイトとドラッグ&ドロップ式管理ビルダー。お客様は店舗ごとのページで予約・お見積り依頼・問い合わせができ、スーパー管理者と店舗管理者がコンテンツを一元管理します。

目次

1. はじめに
 2. セットアップ
 3. 利用者ガイド (来店者・お客様)
 4. 運用者ガイド (店舗管理者)
 5. 管理者ガイド (スーパー管理者)
 6. デプロイガイド
 7. メンテナンスガイド
 8. トラブルシューティング
 9. API リファレンス
 10. 用語集
 11. 変更履歴
 12. サポート・連絡先
-

1. はじめに

1.1 本プロダクトの役割

car-coating は Keeper PRO SHOP のチェーン全店舗を 1 つの Next.js アプリで運営します。提供するものは：

- **コーポレートサイト** (/ ホーム、 /blog ブログ)
- **店舗ごとの公式ページ** (/<slug> 配下に料金、オプション、コーティング、アクセス、レビュー、お知らせ、FAQ、施工事例、予約、お問い合わせ、プライバシーポリシー)
- **管理ダッシュボード** (/admin)。スーパー管理者と店舗管理者が、店舗・コンテンツ・予約・お問い合わせ・バナー・ブログ・KPI・サポートチケットを管理
- **ドラッグ&ドロップ式のページビルダー** (/admin/builder)。ヒーロー / 料金 / ギャラリー / FAQ / バナー / カスタム HTML などの 再利用可能なブロックで店舗ページを構成

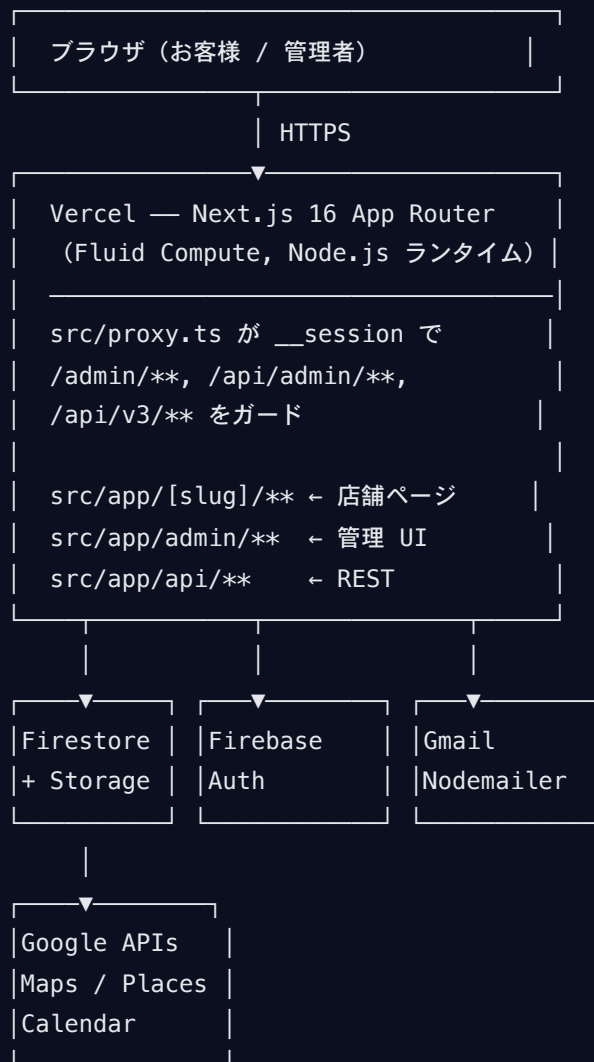
1.2 想定読者

区分	役割	主な導線
お客様	店舗閲覧、見積依頼、予約、問い合わせ	<code>/<slug>/...</code>
店舗管理者	自店舗の予約・問い合わせ・ページ・バナー・お知らせ管理	<code>/admin/**</code> （ <code>managed_stores</code> のみ）
スーパー管理者	全店舗・ユーザー・グローバル設定・マスター・ブログ・診断	<code>/admin/**</code> （フル権限）

1.3 主要な用語

- **Store（店舗）** — 1 つの KeePer PRO SHOP 拠点。URL の `slug` で識別、Firestore の `stores/{store_id}` に保存。
- **Sub-company（サブカンパニー）** — 複数店舗を束ねる親組織。階層・ブランド継承に利用。
- **Sub-store（サブ店舗）** — 親店舗配下の子店舗。 `/<slug>/<subSlug>`。
- **Page layout / Block（ページレイアウト/ブロック）** — 店舗のトップ ページは `page_layout` JSON の型付きブロック列で表現。
- **Banner preset（バナープリセット）** — 再利用可能なバナー素材（構造化 / 生 HTML / パラメータ化テンプレートの 3 モード）。
- **Coating tier（コーティングプラン）** — マスタデータに登録された コーティング商品（クリスタルキーパー、ダイヤモンドキーパー など）。
- **Override / Global default（上書き / グローバル既定）** — 全店共通 値は `globalDefaults` で管理し、各店舗で上書き可能。
- **Reservation（予約）** — 状態は `pending` → `confirmed` → `completed`（または `cancelled`）。
- **Inquiry（お問い合わせ）** — 状態は `open` → `replied` → `closed`。
- **Ticket（チケット）** — 店舗管理者とスーパー管理者間の社内 サポートチケット。

1.4 アーキテクチャ概要



スタック:

層	技術
フレームワーク	Next.js 16 (App Router)、React 19、TypeScript
スタイリング	Tailwind CSS 4
データベース	Firestore (Firebase Admin SDK 経由)
認証	Firebase Auth + 14 日間の <code>__session</code> Cookie
ストレージ	Firebase Cloud Storage
メール	Nodemailer → Gmail SMTP
外部 API	Google Maps (クライアント)、Places (レビュー)、Calendar (予約)
バリデーション	Zod (<code>src/lib/validations.ts</code>)
テスト	Vitest (ユニット) + Playwright (E2E)
デプロイ	Vercel (Fluid Compute, Node.js)

2. セットアップ

2.1 動作要件

- **Node.js** 20 以上 (Next.js 16 要件)
- **npm** 10 以上 (Node 20 同梱)。`package-lock.json` を使用、yarn / pnpm への切替は不可。
- **OS:** macOS / Linux / WSL2
- **対応ブラウザ (公開ページ)** : Chrome / Edge / Firefox / Safari の 直近 2 バージョン。IE11 非対応
- **必要な外部アカウント:**
 - Firebase プロジェクト (Auth + Firestore + Storage 有効化)
 - Gmail アカウント (アプリパスワード発行済)
 - Google Cloud プロジェクト (Maps JS / Places / Calendar API 有効化)
 - (任意) Vercel アカウント (デプロイ用)

2.2 インストール

```
git clone <repo-url> car-coating
cd car-coating
npm install
cp .env.example .env.local
# .env.local を埋める (§2.3 参照)
npm run dev
# → http://localhost:8080
```

2.3 環境変数（初回設定）

`.env.local` がローカル用、本番は Vercel の `vercel env` で管理。正本リストは `.env.example`。

変数	用途
<code>FIREBASE_PROJECT_ID</code>	Firebase Admin SDK プロジェクト ID
<code>FIREBASE_CLIENT_EMAIL</code>	サービスアカウントのメール
<code>FIREBASE_PRIVATE_KEY</code>	サービスアカウント秘密鍵。 <code>\n</code> エスケープを保ったまま貼り付け
<code>NEXT_PUBLIC_FIREBASE_API_KEY</code>	Firebase クライアント SDK（ブラウザ同梱）
<code>NEXT_PUBLIC_FIREBASE_AUTH_DOMAIN</code>	例: <code>your-project.firebaseio.com</code>
<code>NEXT_PUBLIC_FIREBASE_PROJECT_ID</code>	<code>FIREBASE_PROJECT_ID</code> と同じ
<code>NEXT_PUBLIC_FIREBASE_STORAGE_BUCKET</code>	画像アップロード用バケット
<code>NEXT_PUBLIC_FIREBASE_MESSAGING_SENDER_ID</code>	Messaging Sender
<code>NEXT_PUBLIC_FIREBASE_APP_ID</code>	Firebase App ID
<code>NEXT_PUBLIC_GOOGLE_MAPS_API_KEY</code>	Google Maps JS API キー（クライアント）
<code>GOOGLE_PLACES_API_KEY</code>	Places API キー（サーバー）
<code>GOOGLE_OAUTH_CLIENT_ID</code>	OAuth クライアント（ <code>scripts/oauth-create-calendars.ts</code> 用）
<code>GOOGLE_OAUTH_CLIENT_SECRET</code>	OAuth シークレット
<code>GMAIL_USER</code>	送信元 Gmail アドレス
<code>GMAIL_APP_PASSWORD</code>	Gmail アプリパスワード（通常パスワード不可）
<code>NEXT_PUBLIC_SITE_URL</code>	正規ホスト URL（メール / sitemap / OG 用）。既定 <code>http://localhost:8080</code>
<code>NEXT_PUBLIC_LINE_CHANNEL_ID</code>	任意。LINE ボタンを表示

`GMAIL_USER` 未設定時は `src/lib/email.ts` の送信処理が黙って no-op になります（ローカル開発・Playwright で便利）。

2.4 ローカル実行

```

npm run dev          # http://localhost:8080
npm run build         # 本番ビルド（型チェックも兼ねる）
npm run start        # ビルド成果物を配信
npm run lint         # ESLint
npx vitest run       # ユニットテスト
npx playwright test  # E2E（:8081 で dev server 起動）

```

`package.json` に `test:*` ショートカットはありません。 `npx` で直接 起動してください。

2.5 動作確認

1. <http://localhost:8080> を開く。コーポレート LP が表示される。
 2. シード済みの店舗 slug にアクセス（一覧は `npx tsx --env-file=.env.local scripts/list-all-stores.ts`）。
 3. `/login` で `super_admin` カスタムクレーム付き Firebase ユーザーで ログイン。`/admin` に遷移するはず。
 4. 店舗マスター をクリックし、エラーなく一覧が出れば OK。
- 3 で「アカウントが登録されていません」と出る場合、Firebase Auth 上は 存在するが Firestore `users/{uid}` がない状態です (§5.1 参照)。

3. 利用者ガイド（来店者・お客様）

公開 URL の使い方を解説します。

3.1 コーポレートホーム /

LP。ホーム専用ブロック（`hero_home` / `service_menu` / `why_keeper` / `store_finder` / `blog_section` / `news_home` / `process_home` / `cta_home`）で構成、`/admin/homepage` で編集します。

 スクリーンショット: ホーム

3.2 店舗トップ /<slug>

`src/app/[slug]/page.tsx`。表示されるセクションは店舗の `page_layout`（ページビルダーで編集）に従います。標準順は：

`hero` → `store_intro` → `staff_photo` → `before_after` → `gallery` → `usp` → `quiz` → `simulator` → `cases` → `pricing` → `staff` → `concerns` (FAQ) → `news` → `process` → `benefits` → `access` → `cta`。

メモ: 全 active 店舗のトップ URL は `/sitemap.xml` に出力されます。

3.3 店舗のサブページ

URL	内容	実装
<code>/<slug>/access</code>	住所・地図・駐車場・最寄り駅	<code>src/app/[slug]/access/</code>
<code>/<slug>/booking</code>	カレンダー + 予約フォーム	<code>src/app/[slug]/booking/</code>
<code>/<slug>/cases</code>	過去の施工事例ギャラリー	<code>src/app/[slug]/cases/</code>
<code>/<slug>/coatings</code>	コーティングプラン比較	<code>src/app/[slug]/coatings/</code>
<code>/<slug>/guide</code>	ご利用ガイド	<code>src/app/[slug]/guide/</code>
<code>/<slug>/inquiry</code>	お問い合わせフォーム	<code>src/app/[slug]/inquiry/</code>
<code>/<slug>/news</code>	店舗のお知らせ	<code>src/app/[slug]/news/</code>
<code>/<slug>/options</code>	オプション一覧	<code>src/app/[slug]/options/</code>
<code>/<slug>/price</code>	詳細料金表	<code>src/app/[slug]/price/</code>
<code>/<slug>/privacy</code>	プライバシーポリシー	<code>src/app/[slug]/privacy/</code>
<code>/<slug>/reviews</code>	Google Places レビュー	<code>src/app/[slug]/reviews/</code>
<code>/<slug>/<subSlug>[/...]</code>	サブ店舗	<code>src/app/[slug]/[subSlug]/</code>

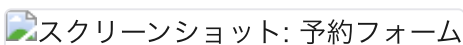
3.4 予約手順

導線: 店舗トップ → **予約 / Book** → `/<slug>/booking`。

1. **日付を選ぶ** — `BookingCalendar.tsx` が `GET /api/slots?store=<id>&month=YYYY-MM` を呼び、空きのない日をグレーアウト。
2. **時間を選ぶ** — 日付を選ぶと `GET /api/slots?store=<id>&date=YYYY-MM-DD` で時間帯を取得。
3. **フォーム入力** (`ReservationForm.tsx`)。必須: 氏名・電話・メール。任意: 車両情報・備考・コーティング選択・オプション選択。
4. **送信** — `POST /api/reservation`。 `type: 'visit'` の場合 `autoConfirm` 既定 `true` で即 `confirmed`。
5. **確認メール** が届く (件名 **【確定】ご予約確認 - <店舗名>**)。 `.ics` 添付付き、キャンセル URL は `/cancel/<id>?token=<cancelToken>`。
6. **キャンセル** はリンクから可能。アカウント不要、 `cancelToken` で認証。

よくある失敗:

- **過去日の予約** — `Booking must be in the future` で 400。
- **メール書式不正** — `^[^\s@]+@[^\s@]+\.[^\s@]+$` を満たさないと弾かれる。
- **レート制限** — IP あたり 5 件 / 分。



3.5 お問い合わせ・見積依頼

導線: 店舗トップ → お問い合わせ → `/<slug>/inquiry`。専用フローは `/estimate`。

1. 氏名・電話（任意）・メール・本文を入力、必要に応じてコース選択。
2. 送信 → `POST /api/inquiry`。
3. お客様には受付メール、店舗側には `storeSettings/{store_id}.notificationEmails` 宛に通知メール。
4. 店舗管理者の `/admin/inquiries` に `open` で表示。

レート制限: IP あたり 10 件 / 分。スタッフが返信したのちのお客様 側追跡には `/api/inquiries/[token]` を使用（セッション不要）。

3.6 レビュー

`/<slug>/reviews` は `GET /api/reviews/[placeId]` をサーバー側で呼び、`GOOGLE_PLACES_API_KEY` を Places API にプロキシ（ブラウザに鍵は 出ません）。

3.7 ブログ

`/blog` は公開記事一覧、`/blog/[articleSlug]` で個別記事。記事は スーパー管理者が `/admin/blog` で投稿します。

4. 運用者ガイド（店舗管理者）

`/login` でログイン。メール+パスワードに加え Google ログインも対応（ただし `users` コレクションに事前登録要、§5.1）。

ログイン後 `/admin` に遷移。サイドバー（`AdminSidebar.tsx`）はロール ごとに表示制御されます。
`store_admin` の場合：

- ダッシュボード — `/admin`。統計とクイックアクション
- 予約管理 — `/admin/bookings`
- お問い合わせ — `/admin/inquiries`
- ページビルダー — `/admin/builder`
- バナーメーカー — `/admin/banners`
- お知らせ管理 — `/admin/news`
- KPIダッシュボード — `/admin/kpi`
- チケット — `/admin/tickets`
- CSVインポート / インポート履歴 — `/admin/stores/import` / `/admin/imports`

店舗管理者はトークンの `managed_stores` に含まれる店舗のみ操作可能。サーバー側で `canManageStore(user, storeId)`（`src/lib/auth.ts`）が強制します。

4.1 予約の確定 — /admin/bookings

1. `pending` の行を選ぶ
2. **承認** をクリック → `PATCH /api/admin/bookings` (`bookingPatchSchema` 検証)
3. 確定処理:
 - ステータスが `confirmed`
 - `storeSettings/{storeId}.calendarId` 設定済なら `createCalendarEvent()` (`src/lib/google-calendar.ts`) で Google カレンダー登録
 - お客様に **【確定】** メール + `.ics`
 - 任意の `adminMessage` がメールに含まれる

その他遷移: `confirmed` → `completed` (来店後)、`*` → `cancelled` (キャンセル + カレンダーイベント削除)。

4.2 お問い合わせ返信 — /admin/inquiries

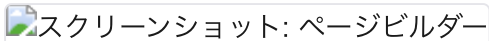
1. `open` の問い合わせを開く
2. 返信本文を入力 → **返信** → `PATCH /api/admin/inquiries` (`inquiryPatchSchema` 検証)
3. お客様にメール (`sendInquiryReplyEmail()`)、ステータスは `replied` に。完了時は **クローズ** で `closed` に。

4.3 店舗ページ編集 — /admin/builder

`@dnd-kit/sortable` を使ったブロック並び替え UI。ブロック型は `src/lib/block-types.ts` 参照。

1. `/admin/builder` で店舗選択 (スーパー管理者)、または `/admin/builder/<storeId>` 直リンク (店舗管理者は自店舗のみ)
2. 左カラムでドラッグして**並び替え**
3. 目アイコンで**表示/非表示** (`block.visible`)
4. ブロックをクリックすると専用エディタ (`src/app/admin/builder/components/editors/`):
 - `hero` — タイトル / 副題 / 画像 / CTA トグル
 - `pricing` — 主要ティア / ぼかし対象 / オプション割引同期
 - `gallery` — カラム数 / 最大枚数
 - `staff` — 見出し + メンバー一覧
 - `concerns` (FAQ) — 質問・回答リスト
 - `custom_html` — 生 HTML/CSS (サーバーで `sanitizeHtml()` / `sanitizeCss()` 通過)
 - `banners` — 構造化 / HTML / テンプレートのバナーリスト
5. **保存** で `page_layout` が JSON 化されて Firestore へ

ロード時に `migrateLayout()` (`src/lib/block-types.ts`) が古い レイアウトに `staff` ブロックを自動挿入し、`concerns` の旧位置を吸収します。



4.4 バナー管理 — /admin/banners

3 モード（`bannerPresetWriteSchema`）：

- **structured** — 見出し / 副題 / 画像 / 元価格 / 割引% / リンク URL / 任意の `custom_css`
- **html** — `html` + `css` の生スニペット
- **combined** — banner-maker のライブプレビュー用、HTML+CSS+JSON パラメータ統合

scope：

- `global` — 全店舗閲覧可、書き込みはスーパー管理者のみ
- `store` — `owner_store_id` の管理者（およびスーパー管理者）が編集可

`is_template: true` のとき、`fields[]`（`text` / `textarea` / `color` / `number` / `select` / `image_url` / `url`）を宣言し、ページビルダーで配置時に キャッシュされた HTML/CSS にパラメータを差し込みます。

4.5 店舗別設定 — /admin/bookings/settings/store

設定可能項目：

- `calendarId` — 確定予約を入れる Google カレンダー ID
- `notificationEmails` — 予約・問い合わせメールの CC 先

`storeSettings/{storeId}` に保存し、5 分間プロセス内キャッシュ（`src/lib/store-settings.ts`）。

4.6 お知らせ・KPI・チケット

- **お知らせ** — `/admin/news`。各エントリ: `title` / `content` / `date` / `visible`。 `news` ブロックでレンダリング。
- **KPI** — `/admin/kpi`。サーバが `kpi/{storeId}/daily/{YYYY-MM-DD}` に問い合わせ・予約・電話タック（`/api/track`）の日次カウンタを記録。
- **チケット** — `/admin/tickets`。スーパー管理者宛に起票。 `ticketActionSchema` で `create` | `reply` | `status` | `edit` | `delete` | `delete_message` を検証。通知メールは `sendTicketNotificationEmail()`。

5. 管理者ガイド（スーパー管理者）

スーパー管理者は全サイドバー項目に加え、`superAdminOnly: true` の項目が表示されます：

- **店舗マスター** — `/admin/stores`
- **店舗構成図** — `/admin/hierarchy`
- **施工事例** — `/admin/cases`
- **トップページ** — `/admin/homepage`

- ・ グローバルデフォルト — `/admin/defaults`
- ・ キャンペーン — `/admin/campaigns`
- ・ ブログ管理 / ブログ CSV — `/admin/blog`, `/admin/blog/import`
- ・ マスターデータ — `/admin/master`
- ・ 診断 — `/admin/diagnostics`

5.1 ユーザー管理 — `/admin/users`

`POST /api/auth/users` (作成)、`DELETE /api/auth/users/[uid]` (削除)。いずれも `requireAuth('super_admin')` でガード。

新規作成手順:

1. `/admin/users` → **新規作成**
2. メール / 仮パスワード / 表示名 / ロール / `managed_stores` (`store_admin` のみ)
3. ハンドラは `createUser()` (`src/lib/auth.ts`) を呼び、
 - Firebase Auth ユーザー作成
 - `{ role, managed_stores }` をカスタムクレームに
 - `users/{uid}` に Firestore 書き込み
4. `/login` でログイン → `/admin` に遷移

初回スーパー管理者のブートストラップ:

```
# Firebase コンソールで Auth ユーザーを作り、
# Firestore へ users/{uid} を書き込み + setUserClaims を実行する
# 1 回限りスクリプトを書いて流す (scripts/seed-* を参考)
npx tsx --env-file=.env.local scripts/seed-all-stores.ts
```

以降は `POST /api/admin/setup` でスーパー管理者を増やせます (同じく `requireAuth('super_admin')` でガード)。

5.2 ロールと権限

操作	super_admin	store_admin
全店舗閲覧	○	managed_stores のみ
店舗作成・削除	○	×
店舗フィールド・レイアウト更新	○	managed_stores のみ
ユーザー作成・編集	○	×
マスタ・グローバル既定・キャンペーン・ホーム・階層	○	×
ブログ編集	○	×
自店舗の予約承認・キャンセル	○	○
自店舗の問い合わせ返信	○	○
自店舗の KPI 閲覧	○	○
チケット起票・返信	○	○

強制点: `src/proxy.ts` が未認証アクセスを拒否、各ハンドラは `requireAuth(...)` で再チェック、Firestore 直接読みは `firestore.rules` で同一ロジックを適用。

5.3 システム設定（スーパー管理者）

ページ	制御内容
/admin/defaults	globalDefaults 文書（テーマ・営業時間・既定値）。override_flags で店舗別上書き
/admin/master	master_data/coating_tiers, master_data/appeal_points。price_overrides JSON でサイズ別上書き
/admin/homepage	コーポレートホームの PageLayout
/admin/campaigns	キャンペーン既定値（campaignDefaultsSchema）
/admin/hierarchy	sub_companies
/admin/diagnostics	読み取り専用ヘルスチェック

5.4 バックアップ・リストア

組み込みバックアップ CLI はありません。推奨:

- Firebase / Google Cloud コンソールで **Firestore 日次バックアップ** を有効化（GCS 出力）
- 軽量スナップショットは `GET /api/v3/stores/export`（CSV 全店舗）。リストアは `POST /api/v3/stores`（Content-Type: text/csv）
- インポート毎に `import_backups/{importId}` に変更前スナップショット を保存。 `POST /api/admin/imports/[importId]/restore` で**ロールバック可**

5.5 ログ

- **Vercel Functions ログ** — 主因。 `vercel logs <deployment-url>` か ダッシュボード
- **Firebase Auth ログ** — ログイン失敗・トークン検証
- 本アプリの API レスポンスは Firebase エラーやスタックトレースを 決して返さず、サーバー側で `console.error` フル出力

5.6 監視

- `@vercel/analytics`、`@vercel/speed-insights` を導入済（自動計測）
 - `/admin/diagnostics` がアプリ内ヘルス
 - 外形監視は `GET /api/admin/diagnostics`（Cookie 無しで 401 が 返れば「アプリは生きている」シグナル）または単に `GET /`
-

6. デプロイガイド

6.1 対応ターゲット

- **Vercel**（本番。Fluid Compute, Node.js）。ビルドは `next build`、Edge ランタイム不使用
- 素の Node 20 + `npm run build && npm run start` でステージングは 動作するが本番ターゲットではありません
- `Dockerfile` は同梱しません

6.2 本番環境変数

§2.3 と同じ。本番固有の注意:

変数	型	既定	未設定時の影響
<code>FIREBASE_PROJECT_ID</code>	string	なし	起動失敗
<code>FIREBASE_CLIENT_EMAIL</code>	string	なし	起動失敗
<code>FIREBASE_PRIVATE_KEY</code>	<code>\n</code> 含む複数行	なし	起動失敗
<code>NEXT_PUBLIC_FIREBASE_*</code>	string	なし	クライアント Firebase が動作不能
<code>NEXT_PUBLIC_GOOGLE_MAPS_API_KEY</code>	string	空	地図白紙
<code>GOOGLE_PLACES_API_KEY</code>	string	空	レビュー空
<code>GMAIL_USER</code> / <code>GMAIL_APP_PASSWORD</code>	string	なし	メール送信が黙って no-op
<code>NEXT_PUBLIC_SITE_URL</code>	URL	<code>https://car-coating.vercel.app</code>	キャンセルリンク・ sitemap・OG が誤ホ スト
<code>GOOGLE_OAUTH_CLIENT_ID</code> / <code>_SECRET</code>	string	なし	カレンダー作成スクリ プト不可
<code>NEXT_PUBLIC_LINE_CHANNEL_ID</code>	string	なし	LINE ボタン非表示

Vercel CLI:

```
vercel env add FIREBASE_PROJECT_ID production
vercel env pull .env.local
vercel env ls production
```

6.3 デプロイ

```
vercel deploy # プレビュー
vercel --prod # 本番
```

6.4 DB マイグレーション手順

Firestore はスキーマレスなのでフィールド追加はほぼ無事故:

1. 既定値付きでコードに追加
2. `src/lib/validations.ts` の Zod を拡張
3. (任意) `scripts/` のバックフィル:

```
npm run tsx --env-file=.env.local scripts/backfill-override-flags.ts
```

4. 複合インデックスが必要ななら `firestore.indexes.json` を更新し:

```
firebase deploy --only firestore:indexes
```

5. アクセス変更時は `firestore.rules` :

```
firebase deploy --only firestore:rules
```

6.5 ゼロダウンタイムデプロイチェックリスト

- [] `npm run lint` クリーン
- [] `npm run build` 成功
- [] `npx vitest run` 成功
- [] `npx playwright test` 成功
- [] Firestore ルール変更ありなら、**先に**寛容めに反映
- [] 新インデックスは**先に**反映（ビルドに数分）
- [] `vercel --prod` を緑コミットから
- [] スモークテスト: `/` , `/<slug>` , `/admin` , `/admin/bookings`

6.6 ロールバック

```
vercel ls  
vercel promote <previous-url>
```

不可逆な Firestore マイグレーションが絡む場合は、§5.4 のエクスポート からリストアしてから戻すこと。

6.7 SSL / ドメイン

ドメインは Vercel ダッシュボードまたは `vercel domains` で管理、SSL は自動。正規ホスト名を変更したら `NEXT_PUBLIC_SITE_URL` を必ず 更新（メール・sitemap・robots・OG が参照）。

7. メンテナンスガイド

7.1 依存更新方針

- **Next.js / React / TypeScript:** マイナー & パッチは月次。メジャー は `node_modules/next/dist/docs/` を読んでから feature-loop で実施
- **Firebase Admin / Client SDK:** マイナー月次。メジャーは認証 リグレーションテスト必須
- **Tailwind CSS 4:** パッチ随時
- **Zod:** パッチのみ無調整 OK、メジャーは `validations.ts` 全面確認

`npm outdated` で見て、小パッチで上げる。Playwright が安全網。

7.2 Firestore メンテ

- 問い合わせ・予約は無制限に増える。四半期ごとのアーカイブを推奨（built-in クリーンアップなし）
- KPI は `kpi/{storeId}/daily/{YYYY-MM-DD}`、O(店舗 × 日数)。年単位 なら問題なし、巨大化時はロールアップを検討
- `firestore.indexes.json` から不要インデックスを定期削除

7.3 ストレージ清掃

- `/admin/upload` の画像は `gs://<bucket>/...` に保存。店舗削除時に 画像は自動削除されません。
`scripts/check-orphan-data.ts` / `scripts/fix-orphan-data.ts` で手動掃除
- `import_backups/{importId}` は小さいが年に 1 度の整理を推奨

7.4 性能チューニング

- `storeSettings` キャッシュ TTL は 5 分。問い合わせ・予約が バーストするなら延長検討
- レート制限は 60 秒ウィンドウ、ルート別 max（予約 5/分、 問い合わせ 10/分、セッション 10/分）。`src/lib/rate-limit.ts` と 各呼び出し点で調整
- `/admin/inquiries` / `/admin/bookings` のページングは現状クライ アント側。1 店舗 5000 件超になったら見直し

7.5 既知の制約

- レート制限は**プロセス内メモリ**。コールドスタートでリセット、Vercel インスタンス間で共有しません。本格運用は Upstash Redis 推奨
 - `sanitizeHtml` はサーバー側、CSS サニタイズはベストエフォート。信用できない人物に管理者ロールを付けないこと
 - 予約カレンダーはブラウザ側で月次空きを数分キャッシュ。他人の確定 予約が一時的に表示残りすることあり
 - 管理 UI にダークモードなし
-

8. トラブルシューティング

症状	原因	対処
ログイン後すぐ <code>/login?redirect=...</code> に戻される	<code>__session</code> Cookie が立たない (Session creation failed)。 <code>FIREBASE_PRIVATE_KEY</code> の <code>\n</code> 崩れ等	環境変数を見直し。秘密鍵は <code>\n</code> エスケープを保つ
ログイン応答に「アカウントが登録されていません」	Firebase Auth にはあるが Firestore <code>users/{uid}</code> がない	<code>/admin/users</code> か <code>POST /api/admin/setup</code> で登録
<code>/api/reservation</code> が <code>Booking must be in the future</code>	サーバ時刻基準で過去日	未来日を指定。Vercel リージョン / 時計を確認
<code>/api/reservation</code> が <code>Rate limit exceeded</code> (429)	同 IP から 5 件/分超過	60 秒待つ
予約確認メールが届かない	<code>GMAIL_USER</code> / <code>GMAIL_APP_PASSWORD</code> 未設定、または Gmail がブロック	環境変数とアプリパスワード再発行。ログの <code>[reservation <id>] email <i> failed</code> 確認
確定時に Google Calendar に登録されない	<code>storeSettings.calendarId</code> 空、またはサービスアカウントがカレンダー招待されていない	<code>scripts/oauth-create-calendars.ts</code> / <code>save-calendar-ids.ts</code> 実行、サービスアカウントに書込権限付与
<code>/admin/...</code> API が 401	<code>__session</code> 期限 (14 日) 切れ、または Admin SDK が拒否	再ログイン、環境変数確認
新規店舗の URL で 404	<code>slug</code> 誤り、または <code>is_active: false</code>	<code>is_active</code> を切替
料金ブロックでティアが正しくない	<code>featured_tier_ids</code> がレガシー ID	ビルダーが <code>crystal-keeper</code> → <code>crystal</code> 等に自動マイグレーション。レイアウトを再保存
サイドバーに項目が足りない	<code>store_admin</code> ログイン中 (<code>storeAdminVisible</code> のみ表示)	<code>super_admin</code> でログイン
Custom HTML ブロックの内容が消える	<code>sanitizeHtml()</code> が disallow タグ・属性を削除	許可タグへ整形。 <code><script></code> は意図的に拒否
<code>/sitemap.xml</code> が古いホスト	<code>NEXT_PUBLIC_SITE_URL</code> 未更新	更新して再デプロイ
<code>/api/reviews/[placeId]</code> 空	<code>GOOGLE_PLACES_API_KEY</code> 未設定、Place ID 誤り	環境変数と <code>google_place_id</code> を確認
バナー HTML モードが空表示	サニタイザ全削除	<code>src/lib/sanitize.ts</code> の許可リストを確認、または structured モード

症状	原因	対処
ビルドが <code>firebase-admin</code> import エラー	ESM/CJS 混在、ロックファイル古い	<code>rm -rf node_modules package-lock.json && npm install</code>

9. API リファレンス

すべて `src/app/api/**`。認証モデル:

- 公開 — 認証なし、IP レート制限あり
- 要認証 — `__session` Cookie 必須 (`POST /api/auth/session` 発行)。 `src/proxy.ts` で先制チェック、各ハンドラで再度 `requireAuth(role?)`
- スーパー管理者専用 — `role === 'super_admin'`

エラー応答は `{ "error": "<message>" }` 形式、4xx/5xx。Firebase 原文は出ません。

9.1 認証

`POST /api/auth/session` — セッション Cookie 発行

公開 (10 件/分/IP)。

```
curl -X POST https://example.com/api/auth/session \
  -H 'Content-Type: application/json' \
  -d '{"idToken": "<firebase-id-token>"}'
```

応答: 200 `{ "success": true }` と `Set-Cookie: __session=...; HttpOnly; Secure; SameSite=Lax; Path=/; Max-Age=1209600`。エラー: 400 Missing idToken、401 Invalid token、403 アカウントが登録されていません、429 Rate limit exceeded。

`DELETE /api/auth/session` — ログアウト

公開。Cookie 削除。200 `{ "success": true }`。

`POST /api/auth/users` — ユーザー作成

スーパー管理者。Body: `{ email, password, displayName, role, managed_stores? }`。

`DELETE /api/auth/users/[uid]` — ユーザー削除

スーパー管理者。

`POST /api/admin/setup` — 追加スーパー管理者作成

スーパー管理者。Body: `{ email, password, displayName? }`。

9.2 公開 API

GET /api/slots?store=<id>&month=YYYY-MM

月次空き。 { "dates": [...] }。

GET /api/slots?store=<id>&date=YYYY-MM-DD

日内空き。 { "slots": [...] }。

POST /api/reservation — 予約作成

公開 (5 件/分/IP)。

Body 例:

```
{
  "type": "visit",
  "storeId": "shinjuku-honten",
  "name": "山田太郎",
  "phone": "09012345678",
  "email": "taro@example.com",
  "notes": "",
  "date": "2026-06-01",
  "time": "10:30",
  "autoConfirm": true,
  "vehicleInfo": "Toyota Prius 2023",
  "selectedCoatings": ["crystal"],
  "selectedOptions": []
}
```

応答: 200 { "id": "<reservationId>" }。エラー: 必須欄ごとの 400、レート 429。副作用: お客様確認メール+ICS、店舗通知メール、 `storeSettings.calendarId` あれば Google カレンダー登録、KPI 加算。

POST /api/cancel/[id]?token=<cancelToken> — 予約キャンセル

公開 (トークン認証)。 `cancelled` / `completed` は幕等で拒否。カレンダー削除 + 通知メール。

POST /api/inquiry — 問い合わせ送信

公開 (10 件/分/IP)。Body: { `storeId`, `name`, `phone?`, `email`, `message?`, `selectedTier?`, `vehicleInfo?` }。応答 200 { "id": "<inquiryId>", "emailWarnings": ["customer-confirmation" | "staff-notification"] }。 `emailWarnings` 配列は確認/通知メールが部分失敗したことをフォームに通知し、フォールバック表示できるようにします。

GET /api/inquiries/[token] — トークン経由の公開ルックアップ

GET /api/reviews/[placeId] — Google Places レビュー

サーバー側プロキシ。 `GOOGLE_PLACES_API_KEY` を使用。

GET /api/blog — ブログ記事一覧（公開）

POST /api/track — 電話タップ・リンククリック計測

`TrackedPhoneLink` / `TrackedLink` から呼ばれ、KPI を更新。

9.3 V3 店舗 API

ルート	メソッド	ロール
<code>/api/v3/stores</code>	GET	公開（active のみ）
<code>/api/v3/stores?all=true</code>	GET	要認証（inactive 含む）
<code>/api/v3/stores</code>	POST	super_admin（JSON or <code>text/csv</code> 、最大 <code>MAX_BATCH_STORES = 500</code> ）
<code>/api/v3/stores/[storeId]</code>	GET / PUT / DELETE	super_admin
<code>/api/v3/stores/export</code>	GET	super_admin
<code>/api/v3/sub-companies</code>	GET / POST / PUT / DELETE	super_admin
<code>/api/v3/campaign</code>	POST	super_admin

検証: `v3StoreWriteSchema` / `v3StorePartialSchema` / `campaignDefaultsSchema`。

9.4 管理 API

すべて `/api/admin/**`。proxy が未認証を弾き、ハンドラが `requireAuth(role)` で再チェック。

ルート	メソッド	ロール	スキーマ
/api/admin/bookings	GET, PATCH	要認証	bookingPatchSchema
/api/admin/inquiries	GET, PATCH	要認証	inquiryPatchSchema
/api/admin/tickets	GET, POST	要認証	ticketActionSchema
/api/admin/tickets/count	GET	要認証	—
/api/admin/tickets/export	GET	super_admin	—
/api/admin/store-settings	GET, PUT	要認証（管理店舗）	—
/api/admin/upload	POST	要認証（管理店舗）	multipart
/api/admin/banner-presets	GET, POST	要認証	bannerPresetWriteSchema
/api/admin/banner-presets/[id]	GET, PUT, DELETE	scope 別	bannerPresetWriteSchema
/api/admin/blog	GET, POST	super_admin	blogPostWriteSchema
/api/admin/blog/[postId]	GET, PUT, DELETE	super_admin	blogPostWriteSchema
/api/admin/blog/import	POST	super_admin	CSV
/api/admin/defaults	GET, PUT	super_admin	—
/api/admin/defaults/policy	GET, PUT	super_admin	—
/api/admin/defaults/overriding-stores	GET	super_admin	—
/api/admin/master/coating-tiers	GET, PUT	super_admin	—
/api/admin/master/appeal-points	GET, PUT	super_admin	—
/api/admin/homepage	GET, PUT	super_admin	—

ルート	メソッド	ロール	スキーマ
/api/admin/sub-companies	GET, POST, PUT, DELETE	super_admin	—
/api/admin/seed-content	POST	super_admin	—
/api/admin/template	GET, POST	super_admin	—
/api/admin/custom-html-notice	POST	super_admin	—
/api/admin/diagnostics	GET	super_admin	—
/api/admin/kpi	GET	要認証（管理 店舗）	—
/api/admin/kpi/export	GET	super_admin	CSV
/api/admin/overrides	POST	super_admin	—
/api/admin/revalidate	POST	super_admin	—
/api/admin/stores	GET	要認証	—
/api/admin/stores/[storeId]	GET, PUT, DELETE	scope 別	—
/api/admin/stores/import	POST	super_admin	CSV
/api/admin/stores/import/template	GET	super_admin	—
/api/admin/imports/history	GET	super_admin	—
/api/admin/imports/[importId]	GET	super_admin	—
/api/admin/imports/[importId]/download	GET	super_admin	—
/api/admin/imports/[importId]/restore	POST	super_admin	—
/api/admin/setup	POST	super_admin	—

9.5 例: 予約確定

```
curl -X PATCH https://example.com/api/admin/bookings \  
-H 'Cookie: __session=...' \  
-H 'Content-Type: application/json' \  
-d '{  
  "reservationId": "abc123",  
  "status": "confirmed",  
  "confirmChoiceIndex": 0,  
  "adminMessage": "ご来店をお待ちしております。"  
}'
```

応答 200 { "success": true }。エラー: 400 (Zod) / 401 (Cookie 無し) / 403 (権限なし) / 404 (予約なし)。

10. 用語集

用語	意味
App Router	Next.js のルーティング。 <code>src/app/</code> ベース、旧 <code>pages/</code> の置き換え
Block（ブロック）	店舗ページに描画される型付きコンテンツ単位
Block layout / Page layout	単一ページのブロック並びを表す JSON
Coating tier	マスターに登録されたコーティング商品
Cancel token	予約ごとのランダム UUID。 <code>/api/cancel/[id]?token=...</code> で本人確認
Custom claim	Firebase ユーザーのメタ情報（ <code>role</code> / <code>managed_stores</code> ）
Firestore	Google の NoSQL ドキュメント DB
Global default / Override	全店共通既定値と店舗別上書き
Inquiry	問い合わせ（ <code>open</code> / <code>replied</code> / <code>closed</code> ）
KPI	店舗ごとの日次指標（電話タップ・問い合わせ・予約）
Page builder	<code>/admin/builder</code> のドラッグ&ドロップ UI
Preset	バナー素材（ <code>scope</code> , <code>mode</code> , optional <code>is_template</code> ）
Proxy	Next.js 16 の改名版 middleware。 <code>src/proxy.ts</code>
Reservation	予約（ <code>pending</code> / <code>confirmed</code> / <code>completed</code> / <code>cancelled</code> ）
Sanitizer	<code>src/lib/sanitize.ts</code> / <code>sanitize-css.ts</code> 。管理者作成の HTML/CSS を必ず通す
Session cookie	<code>__session</code> 、httpOnly+secure+SameSite=lax、14 日 TTL
Store	KeePer PRO SHOP 拠点。Firestore 1 文書
Sub-company	親組織。階層・ブランド継承用
Sub-store	<code>/<slug>/<subSlug></code> に置く子店舗
Super admin / Store admin	2 ロール。スーパー管理者は全権、店舗管理者は <code>managed_stores</code> のみ
Ticket	店舗管理者 ↔ プラットフォーム間の社内チケット

11. 変更履歴

バージョン	日付	概要
0.1.0	2026-05-07	マニュアル初版スナップショット

<!-- TODO: リリースタグ毎に追記 -->

12. サポート・連絡先

<!-- TODO: サポートメール（例: support@your-domain.jp）を追記 --> <!-- TODO: 課題管理 URL（例: GitHub Issues）を追記 --> <!-- TODO: Slack / 代替連絡経路を追記 -->

社内向けのサポートは、店舗管理者が `/admin/tickets` から起票できます。プラットフォーム側は `sendTicketNotificationEmail()` で通知を受領します。